Concurrent-Slot Non-Determinism in the llama.cpp Reasoning Server

InspectAI / khidmah-nl2sql-prototype — llama.cpp self-hosting migration, investigation notes

Component	Reasoning + turn_classifier model server (`qwen3.5-9b`, Q4_K_M, llama.cpp `llama-server`)
Host	`piserve-rtx5080` (10.77.20.13), single RTX 5080, 16303 MiB VRAM
Date investigated	2026-08-17
Status	Resolved — reasoning server reverted to `-np 1`; determinism restored and confirmed

1. Summary

As part of self-hosting InspectAI's reasoning model on llama.cpp (moving off Ollama/vLLM — see "Why earlier hosting attempts didn't pan out" in the accompanying setup plan), the reasoning server was briefly run with two concurrent slots (`-np 2`) to reduce KV-cache prefix contention between the main reasoning loop and the turn_classifier (which reuses the same model/role with a different, shorter prompt). While reviewing real query logs for SQL correctness, an identical question asked twice in the same conversation produced two different results — one correct, one wrong. Controlled testing directly against the model server (bypassing the application entirely) confirmed this was caused by `llama-server`'s concurrent-request batching introducing genuine floating-point non-determinism, even at `temperature=0`. The reasoning server was reverted to a single slot (`-np 1`), which was confirmed via the same test to restore full determinism.

2. How it was discovered

A user asked "Show all audit cycles" at two points in the same conversation. The correct answer — 6 active audit cycles, no status filter — had already been returned once. On the repeat, the SQL-generation model produced a different, incorrect query:

Turn	Generated SQL	Result
Earlier (correct)	`SELECT ... FROM audit_cycle acyc WHERE acyc.active = TRUE ...`	6 rows — correct
Later (incorrect)	`SELECT ..., acyc.due_date, ..., acyc.due_date, ... WHERE acyc.active = TRUE`	0 rows — wrong (fabricated status filter never asked for; duplicated

	`AND acyc.status = 'ONGOING'`	`due_date` column)

Both calls retrieved **identical** few-shot examples from the retrieval layer (confirmed by comparing `example_keys` in the debug logs byte-for-byte), which ruled out a retrieval/prompt-content bug. The only remaining explanation was that the reasoning model itself produced a subtly different draft SQL the second time — despite both roles being explicitly configured at `temperature=0.0`.

3. Controlled investigation

To isolate the cause from any application-level logic (retrieval, conversation state, prompt construction), the test was run directly against `llama-server` via `curl` on the host itself — no application code involved.

Test setup Fixed prompt, fixed sampling parameters

```
POST http://127.0.0.1:10000/v1/chat/completions
{
  "model": "qwen3.5-9b",
  "temperature": 0,
  "max_tokens": 200,
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "List all active audit cycles. Respond with a single SQL query
only, no explanation."}
  ]
}
```

Result A Sequential requests (baseline)

20 identical requests, one at a time, waiting for each to complete before sending the next.

Runs	Distinct outputs	Result
20	1	**100% deterministic** — every response byte-identical

Result B Concurrent requests at `-np 2`

Same 20 requests, fired 2-at-a-time in parallel (`xargs -P 2`), matching the server's slot count.

Output variant	Count
`SELECT * FROM audit_cycles WHERE status = 'active';`	11 / 20

`SELECT cycle_id, cycle_name, start_date, end_date, created_at FROM audit_cycles WHERE status = 'active';`	9 / 20

> **~45/55 split into two genuinely different completions**, from the identical prompt, at `temperature=0`, purely as a function of how many other requests happened to be in flight at the same instant.

Result C — Concurrent requests after reverting to `-np 1`

Identical concurrent test (`xargs -P 2`, 20 requests) repeated after restarting the server with a single slot.

Runs	Distinct outputs	Result
20	1	**100% deterministic** — even though requests were still fired concurrently client-side, they queue and process one-at-a-time server-side with only one slot available

4. Root cause

`temperature=0` guarantees *greedy decoding* — always selecting the highest-scoring token (the argmax over logits). It does **not** guarantee that the logits themselves are computed identically every time.

`llama-server`'s `-np` (`--parallel`) option exists specifically to batch multiple in-flight requests' forward passes together into a single GPU kernel call, for throughput efficiency — that is its entire purpose. GPU floating-point matrix multiplication is not strictly associative: the order in which values are summed/reduced during a batched computation depends on the shape of that batch. When a request is processed alone versus bundled with another request's tokens in the same kernel call, the arithmetic path differs slightly, producing tiny (last few bits of precision) differences in the resulting logits.

Most of the time this is invisible — the top-scoring token has a comfortable margin over the runner-up, and a rounding difference in the sixth or seventh decimal place changes nothing. Occasionally, however, two candidate tokens are close enough in score that this batching-induced noise is enough to flip which one wins the argmax. Because generation is autoregressive (every token conditions on everything generated so far), a single flipped token early in the response cascades into a fully different completion by the end — exactly what was observed in Result B.

> **Isolated specifically to slot concurrency, not KV cache quantization.** The server ran with quantized (`q8_0`) KV cache in *both* the deterministic (`-np 1`) and divergent (`-np 2`) tests — only the slot count changed between Results A/C and Result B. Quantization may be a minor contributing factor when combined with concurrency, but it is not sufficient on its own: `-np 1` with the same quantized cache was fully deterministic.

5. Ruled out

- **Turn_classifier using a different temperature.** Confirmed false by code inspection — `turn_classifier.py` calls `self.reasoning_llm.chat(messages)` directly, reusing the exact same client object (`temperature=0.0` baked in at construction), with no per-call override. Also ruled out by the test methodology itself: the curl-based reproduction never invoked turn_classifier or any application code at all, and still diverged under `-np 2`.
- **A missing determinism flag.** Checked the installed `llama-server` build directly:

  ```
  $ ./build/bin/llama-server --help | grep -i determin
  (no output)
  ```

  No `--deterministic` or batch-invariance flag exists in this build. The only determinism-adjacent flag present is `--seed` (RNG seed for stochastic sampling), which does not apply here — at `temperature=0` there is no sampling randomness to seed against; the divergence is numerical, not stochastic.

6. Decision

The reasoning server was reverted from `-np 2 -c 40000` to `-np 1 -c 20000` (its original configuration). This codebase already treats output reproducibility as a hard requirement — `factory.py`'s `get_reasoning_llm` docstring documents a prior incident (2026-07-30) where sampling-driven variance made eval runs non-comparable across identical question sets, which is why both the reasoning and SQL roles are set to `temperature=0.0` in the first place. The concurrent-batching non-determinism uncovered here is the same class of problem via a different mechanism, and was judged to outweigh the prefix-cache separation benefit that motivated trying `-np 2`.

The SQL generation server (`qwen2.5-coder-7b`, port 10001) was never changed — it ran at `-np 1` throughout, since nothing in the application contends for it the way turn_classifier and the reasoning loop contend for the reasoning role.

Current live configuration

```
# Reasoning + turn_classifier model — port 10000
./build/bin/llama-server \
  -m "/home/ai-user/model-serving/models/qwen3.5-9b-gguf/Qwen_Qwen3.5-9B-Q4_K_M.gguf" \
  -c 20000 \
  -np 1 \
  -ngl 99 \
  -fa on \
  --cache-type-k q8_0 --cache-type-v q8_0 \
  --host 0.0.0.0 \
  --port 10000
```

```
# SQL generation model — port 10001 (unchanged throughout)
./build/bin/llama-server \
  -m "/home/ai-user/model-serving/models/qwen2.5-coder-7b-gguf/qwen2.5-coder-7b-instruct-
q4_k_m.gguf" \
  -c 16384 \
  -np 1 \
  -ngl 99 \
  -fa on \
  --cache-type-k q8_0 --cache-type-v q8_0 \
  --host 0.0.0.0 \
  --port 10001
```

7. If revisited later

- There is ~4.6GB of free VRAM headroom on the card even at `-np 2` on reasoning, so a future attempt at multi-slot concurrency is not VRAM-blocked — it is blocked by this determinism finding specifically.
- If concurrent throughput becomes a real requirement, options worth evaluating before re-enabling `-np 2` on the reasoning role: routing turn_classifier to a *separate* lightweight model/server instead of sharing the reasoning role (removes the prefix-cache contention problem `-np 2` was originally meant to solve, without reintroducing batching non-determinism), or accepting the divergence rate for high-throughput, low-stakes traffic while keeping a separate `-np 1` path for anything eval/reproducibility-sensitive.
- Check whether upstream llama.cpp has since added a genuine batch-invariant/deterministic inference mode — none existed in the build tested here (2026-08-17), but this is an active area of research in LLM-serving frameworks generally.

Investigation conducted via direct `curl` testing against `llama-server` on `piserve-rtx5080`, cross-referenced against `debug_llm_*.log` application logs and source inspection of `backend/src/agent/orchestrator/turn_classifier.py` and `backend/src/llm/factory.py`. Full setup plan and VRAM sizing data: see the accompanying llama.cpp hosting plan document.